

Concurrent Task Dispatcher Report

Introduction

This Rust program models a concurrent task dispatcher similar to an OS scheduler. It generates a mix of CPU-heavy and IO-like tasks and tries to run them efficiently while respecting a simulated CPU capacity limit. The program can run in two workload modes: a default mix where most tasks are IO (about 70%) and fewer are CPU (about 30%), and an alternate mode with an even more IO-heavy distribution (about 80% IO, 20% CPU).

Policies

Two scheduling strategies are compared:

FIFO: The dispatcher prioritizes CPU work before IO work. Because CPU tasks consume a large share of the available CPU budget (around 35% each), the system can reach the 100% cap quickly. Once near the cap, dispatching stalls, causing tasks to wait longer and queues to grow.

Optimized: The dispatcher prefers IO work first and falls back to CPU work afterward. Since IO tasks use much less of the CPU budget (around 10%), the dispatcher can often keep workers active without hitting the cap as frequently. This typically reduces queue buildup and lowers average waiting time.

Main Components

The program is organized into these parts:

Generator: Produces a fixed number of tasks at regular intervals and randomly chooses CPU vs IO according to the selected workload mix. Tasks are sent to the dispatcher via a channel.

Dispatcher/Manager: Receives tasks, keeps separate queues for CPU and IO, assigns tasks to idle workers based on the chosen policy, and enforces the 100% CPU cap by tracking how much CPU is currently “in use.”

Workers (thread pool): A set number of worker threads wait for tasks. When a task arrives, a worker simulates running it by sleeping for the task duration, then reports completion back to the dispatcher. A shutdown message ends the worker loop.

Monitor: Periodically (every 10ms) samples shared status like simulated CPU usage and number of active workers, storing these samples for later analysis.

Metrics: Collects completion reports and monitor samples. At the end, it computes and prints summary stats like total runtime, average wait/turnaround, average/peak CPU usage, and average number of busy workers.

Main: Reads the CLI option for workload mode, runs both policies (FIFO and Optimized), prints per-run summaries, then prints a side-by-side comparison.

Limitations

The main bottleneck is the global CPU cap. CPU tasks consume a large portion of the budget while IO tasks consume much less, so scheduling decisions focus on keeping the simulated CPU load under 100%.

Shared State and Synchronization

Small pieces of “live” status (like current CPU usage and number of busy workers) are shared across threads using atomic variables so the monitor can sample them cheaply. Larger shared data structures, like the metrics vectors, are protected with `Arc<Mutex<...>>` so multiple threads can safely append results. Most coordination happens through channels: the generator sends tasks to the dispatcher, the dispatcher sends work or shutdown messages to workers, and workers send completion reports back. This keeps task assignment centralized and reduces direct shared access to scheduling queues.

Results

Using seed 1, both workloads show the same general outcome: the Optimized (IO-first) policy significantly improves responsiveness and slightly improves total completion time under the CPU cap. Runtime improves modestly, but average wait and turnaround times drop a lot because IO-first scheduling prevents long queue buildup. Peak CPU still reaches 100%, while average CPU stays around the mid-70% range. The number of active workers remains well below the total worker count, indicating the CPU cap—not worker availability—is usually what prevents more parallelism. As a result, the policy mostly reduces queueing delays rather than fully utilizing all workers.

Issues/Bugs Encountered

Most problems during development were due to refactoring and ownership rules in Rust while adding concurrency. One common issue was moving a completion report into the metrics collection and then trying to read fields from it afterward, which Rust disallows; the fix was to copy out needed fields before moving the struct. Another issue came from inconsistent time representations (Duration vs Instant), which broke timing math until timestamps were standardized. Finally, after cleanup, the build failed briefly due to leftover references to an old monitoring type; removing those references and relying on the actual monitor samples fixed it.